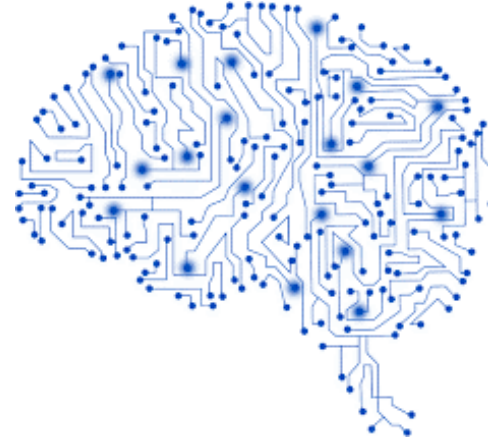




University of Minho
School of Engineering



Dados e Aprendizagem Automática

Quality Metrics and Model Validation

DAA 2025/2026

Filipa Ferraz, Victor Alves, Dalila Durães, Francisco Marcondes, Guilherme Barbosa

Part II

Contents

2

- A Machine Learning Algorithm (Decision Trees)
- Quality Metrics
- Model Validation
- Hands On



Decision Tree

A Machine Learning Algorithm

4

An example of a Machine Learning algorithm is **Decision Tree** (DT)

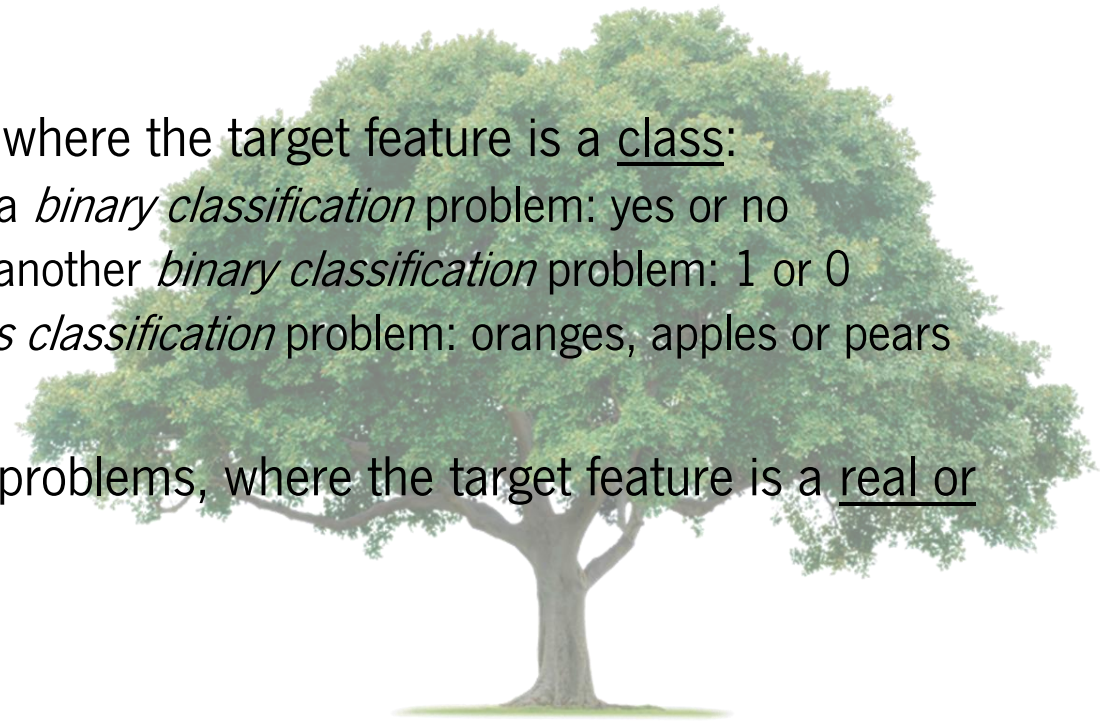
- A DT is a model that predicts the value of a target feature:
 - It is a hierarchical graph (a tree)
 - Each branch represents a selection from a set of alternatives
 - Each leaf node represents a class

A **DT Classifier** is used for classification problems, where the target feature is a class:

- For example, deciding whether to have lunch is a *binary classification* problem: yes or no
- Whether or not someone survived the Titanic is another *binary classification* problem: 1 or 0
- Classifying a set of images of fruit is a *multiclass classification* problem: oranges, apples or pears

Conversely, a **DT Regressor** is used for regression problems, where the target feature is a real or continuous value:

- For example, predicting the traffic flow in km/h
- Predicting stock prices in €





Decision Tree Classifier

Implementing a Decision Tree Classifier: data loading

6

We are going to use the **Titanic dataset**. First, we load the dataset:

```
'''  
Load CSV  
'''  
df = pd.read_csv('titanic_dataset.csv')
```

```
'''  
Inspect dataset  
'''  
df.columns
```

```
Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',  
      'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'],  
      dtype='object')
```

Implementing a Decision Tree Classifier: defining X and y

7

Now we define our input and our target features:

- **X** is typically used to represent the **input**
- **y** to represent the **target**

```
X = df.drop(['Survived'], axis=1)    #input features - everything except the Survived feature
y = df['Survived'].to_frame()       #target feature
```

X											y		
PassengerId	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	Survived		
0	1	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S	0	0
1	2	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C	1	1
2	3	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S	2	1
3	4	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S	3	1
4	5	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S	4	0

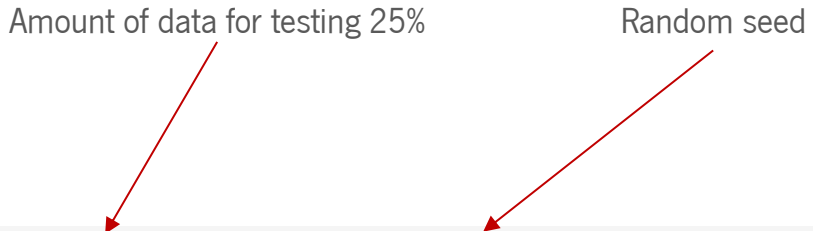
891 rows × 11 columns

891 rows × 1 columns

Implementing a Decision Tree Classifier: train/test split

There are 891 rows of data for both the X and the Y, which corresponds to the entire dataset.
Our next step is to **set aside a small portion of the data to test and validate the model** (25%):

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=2021)
```



```
print("The shape of X %s. X_train has shape %s while X_test has shape %s" %(X.shape, X_train.shape, X_test.shape))
```

```
The shape of X (891, 11). X_train has shape (668, 11) while X_test has shape (223, 11)
```

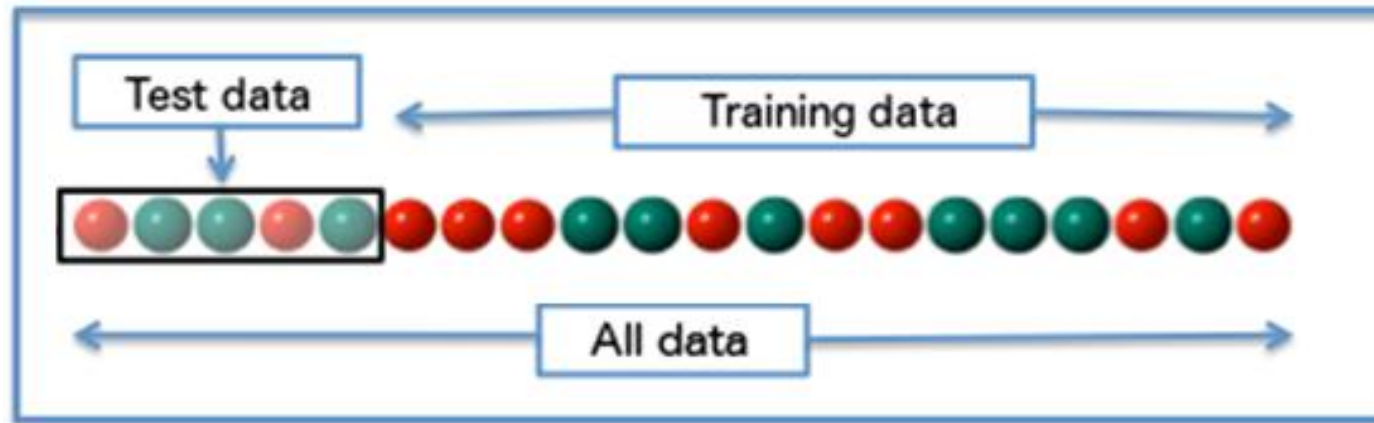
```
print("The shape of y %s. y_train has shape %s while y_test has shape %s" %(y.shape, y_train.shape, y_test.shape))
```

```
The shape of y (891, 1). y_train has shape (668, 1) while y_test has shape (223, 1)
```

Hold-out Validation

9

In essence, we will validate the model using a “partitioning method” to split the training and testing data **once**, i.e. we will **validate the model on unseen data**. This means we **hold-out a subset of data** for testing (80/20, 75/25, 65/35, etc.).



Implementing a Decision Tree Classifier: model fitting

10

How can we now use a DT to **predict whether a passenger would survive the disaster?**

First, we **create an instance** of the classifier, and then we use the **fit function**:

```
clf = DecisionTreeClassifier(random_state=2021)
```

```
clf.fit(X_train, y_train)
```

Implementing a Decision Tree Classifier: model fitting

11

Ups!

```
clf = DecisionTreeClassifier(random_state=2021)
```

```
clf.fit(X_train, y_train)
```

```
-----  
ValueError                                Traceback (most recent call last)  
/tmp/ipykernel_3833/2168925757.py in ?()  
      1 #Training, i.e., fitting the model (using the training data!!)  
----> 2 clf.fit(X_train, y_train)
```

```
ValueError: could not convert string to float: 'Ali, Mr. William'
```

Sklearn decision trees **do not handle categorical data** (see issue #12866)!
(<https://github.com/scikit-learn/scikit-learn/pull/12866>)

Implementing a Decision Tree Classifier: model fitting

12

We could use techniques such as **Label Encoding** or **One-Hot Encoding** to handle categorical data. For now, we will simply **drop** those features:

```
X_train = X_train.drop(['Name', 'Sex', 'Age', 'Ticket', 'Cabin', 'Embarked'], axis=1)
X_test = X_test.drop(['Name', 'Sex', 'Age', 'Ticket', 'Cabin', 'Embarked'], axis=1)
```

```
clf.fit(X_train, y_train)
```

▼ DecisionTreeClassifier

```
DecisionTreeClassifier(random_state=2021)
```

Implementing a Decision Tree Classifier: predictions

13

Now that the model has been fitted, we can use the **predict function** to obtain the survival prediction for each observation (row) in the test set (0 – doesn't survive; 1 – survives):

```
predictions = clf.predict(X_test)
predictions
```

```
array([0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 0,
       1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0, 1, 0,
       0, 1, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1,
       0, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 1, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0,
       0, 1, 1, 0, 0, 1, 0, 1, 1, 1, 0, 0, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1,
       0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1,
       1, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0,
       1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,
       1, 1, 0])
```

```
print(y_test.values.shape)
```

```
(223, 1)
```

We now have predictions for the test set, and we know the actual survival value, as this is stored in the **y_test** variable. How can we **evaluate** our classification model? There are several options.



Quality Metrics

Quality Metrics and Model Evaluation

15

Why are **quality metrics** so important? How else can we **quantify the performance of a model**?

Metrics are used to monitor and measure model performance. Examples include Mean Absolute Error, Mean Squared Error, Accuracy and F1-Score.

However, the choice of metric **depends on the problem at hands**. Is it a classification problem? Or a regression one? Or is it a time series problem?



Quality Metrics

16

Here are some basic functions and classes that will be need at some point in the future:

```
sklearn.metrics.accuracy_score  
sklearn.metrics.auc  
sklearn.metrics.mean_absolute_error  
sklearn.metrics.mean_squared_error  
sklearn.metrics.root_mean_squared_error  
sklearn.metrics.f1_score  
sklearn.metrics.make_scorer  
...
```

Imports

17

Here are some of the imports that may required:

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import confusion_matrix
from sklearn.metrics import recall_score
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import roc_auc_score
from sklearn.metrics import roc_curve
from sklearn.metrics import f1_score
from sklearn.metrics import fbeta_score
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import root_mean_squared_error
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

from sklearn.model_selection import cross_val_score
```

Quality Metrics

18

Some examples:

- Confusion Matrix
- Accuracy
- Precision
- Recall
- ROC
- AUC
- F1 Score
- $F\beta$ Score
- MAE
- MSE
- RMSE

Classification Models: confusion matrix

19

A **confusion matrix** is a table that is used to describe the performance of a **classification model** on a set of test data for which the true values are known.

		True Class	
		Positive	Negative
Predicted Class	Positive	TP	FP
	Negative	FN	TN

Classification Models: accuracy

20

Accuracy is defined as the ratio of correct predictions to the total number of observations.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

		True Class	
		Positive	Negative
Predicted Class	Positive	TP	FP
	Negative	FN	TN

Classification Models: precision and recall

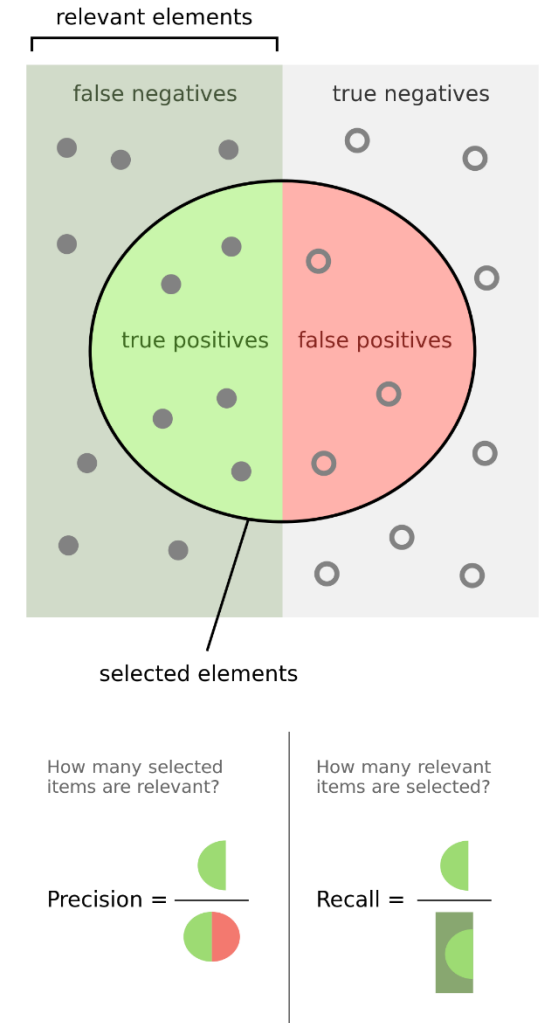
21

Precision (also known as Sensitivity) is a measure of **exactness** and determines the proportion of relevant items among those retrieved.

$$Precision = \frac{TP}{TP + FP}$$

Recall (also known as Specificity) is a measure of **completeness** and determines the fraction of relevant items obtained.

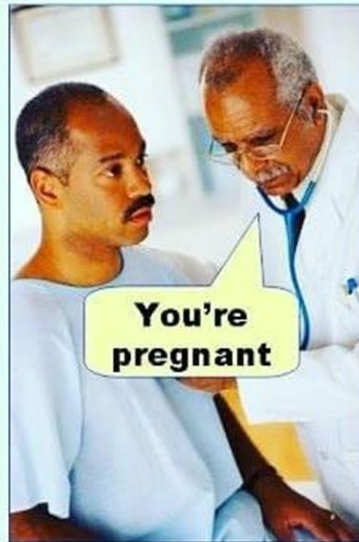
$$Recall = \frac{TP}{TP + FN}$$



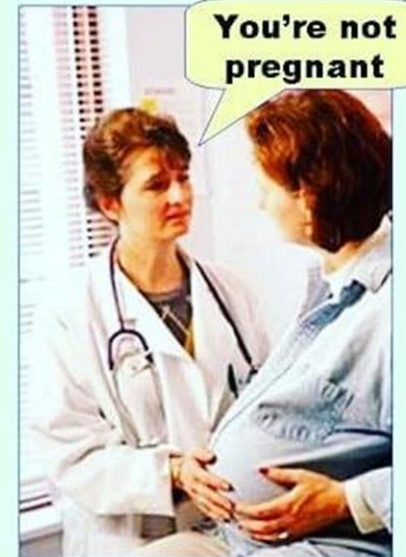
Classification Models: precision and recall

22

Type I error
(false positive)



Type II error
(false negative)



Confusion Matrix-based Metrics

23

Obtaining the **confusion matrix** is simple :

```
confusion_matrix(y_test, predictions)

array([[96, 39],
       [43, 45]])
```

The same applies to the model's **accuracy**, **precision** and **recall**:

```
accuracy_score(y_test, predictions)
```

```
0.6322869955156951
```

```
precision_score(y_test, predictions)
```

```
0.5357142857142857
```

```
recall_score(y_test, predictions)
```

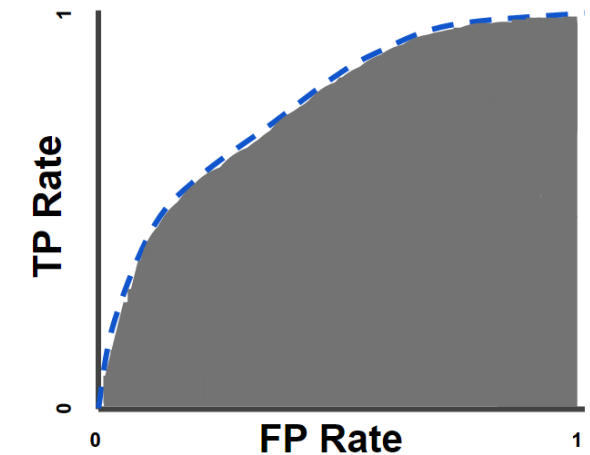
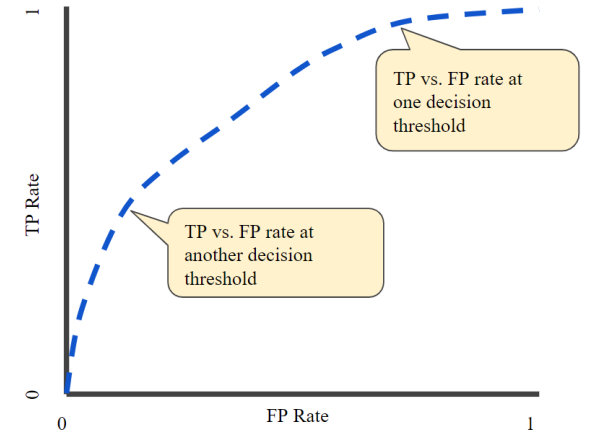
```
0.5113636363636364
```

ROC and AUC

24

The **Receiver Operating Characteristics (ROC)** curve shows how a classification model performs at different classification thresholds. Lowering the threshold means more items are classified as positive, thus increasing both False Positives and True Positives.

The **Area Under the Curve (AUC)** is the two-dimensional area underneath the ROC curve (think integral calculus). Rather than measuring the absolute values of predictions, it measures how well they are ranked, and ranges from 0 to 1. A model whose predictions are 100% incorrect has an AUC of 0, while one whose predictions are 100% correct has an AUC of 1.



ROC and AUC

25

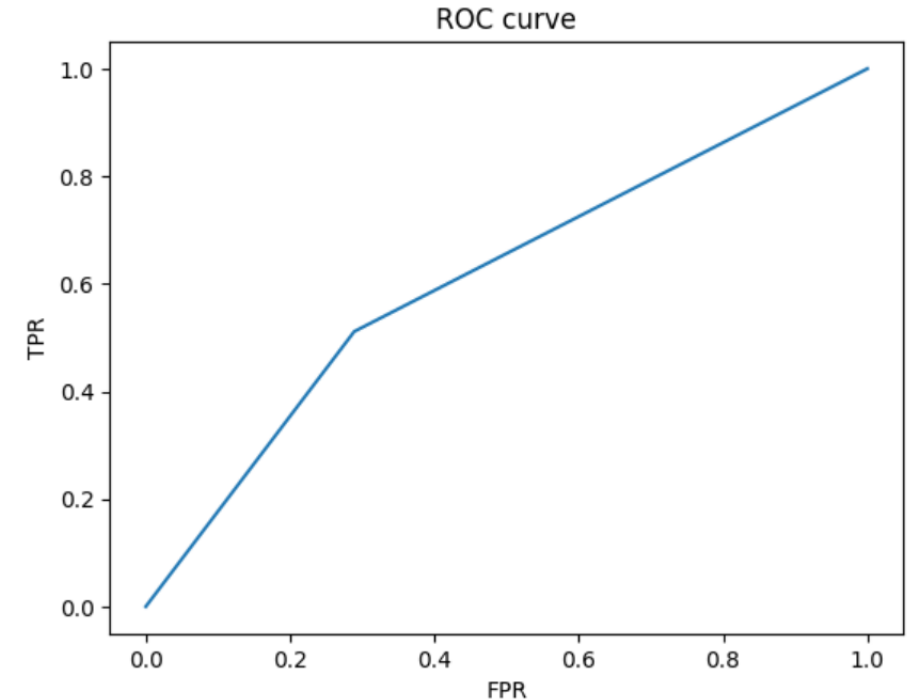
For the **ROC** and the **AUC**:

```
roc_auc_score(y_test, predictions)
```

0.6112373737373737

```
fpr, tpr, _ = roc_curve(y_test, predictions)
```

```
plt.clf()  
plt.plot(fpr, tpr)  
plt.xlabel('FPR')  
plt.ylabel('TPR')  
plt.title('ROC curve')  
plt.show()
```



F_1 and F_β Scores

26

The **F_1 Score** combines precision and recall into a single value for comparison purposes. It can be used to obtain a more balanced view of performance.

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

The F_1 Score gives equal weight to precision and recall. Use the **F_β Score** to weight recall by a factor of β ; with $\beta=1$, the **F_1** and **F_β Scores** are equivalent.

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{Precision} \cdot \text{Recall}}{\beta^2 \cdot \text{Precision} + \text{Recall}}$$

F_1 and F_β Scores

27

For the **F_1** and **F_β Scores:**

```
f1_score(y_test, predictions)
```

```
0.5232558139534884
```

```
fbeta_score(y_test, predictions, beta=0.5)
```

```
0.5306603773584905
```



Decision Tree Regressor

Decision Tree Regressor

29

Let's consider the possibility of **predicting the FARE** paid by those who went to the Titanic. To do that, we would need a **Decision Tree Regressor**.



Implementing a Decision Tree Regressor

30

First, we need to redefine our **input** (X) and **target** (y) features:

```
X = df.drop(['Fare'], axis=1)    #input features - everything except the Fare feature
y = df['Fare'].to_frame()       #target feature
```

We also need to **hold-out some data for testing** and **drop the categorical features** again:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=2021)
X_train = X_train.drop(['Name', 'Sex', 'Age', 'Ticket', 'Cabin', 'Embarked'], axis=1)
X_test = X_test.drop(['Name', 'Sex', 'Age', 'Ticket', 'Cabin', 'Embarked'], axis=1)
```

Implementing a Decision Tree Regressor

31

Now, we simply **fit** and **predict** the **fares** paid by people in the test set:

```
#Training, i.e., fitting the model (using the training data!!)  
clf.fit(X_train, y_train)
```

```
▼      DecisionTreeRegressor  
DecisionTreeRegressor(random_state=2021)
```

```
#obtaining predictions  
predictions = clf.predict(X_test)  
predictions
```

```
array([ 18.7875,   7.8958,  10.5   ,  27.9   ,  34.375 ,  24.15   ,  
       211.5   ,   7.7417,  12.35   ,   7.2292,   7.2292,  52.     ,  
       15.5   ,  18.7875,  13.     ,   8.6625,   8.05   ,  52.     ,  
        8.6625,  35.5   ,  76.2917,  15.5   ,   7.6292,   8.4042,  
        8.6625,   7.125 ,   7.775 ,   7.65   ,  93.5   ,  26.     ,  
        7.8792,   8.6625,  39.6875,  16.1   , 113.275 ,  12.35   ,
```

We now have fare predictions for the test set, and we know the actual fare value, which is stored in the **y_test** variable. How can we **evaluate** our regression model?



Quality Metrics

Mean Absolute Error

33

The **Mean Absolute Error (MAE)** measures the average magnitude of the errors in a set of predictions without considering their direction.

$$MAE = \frac{1}{n} \sum_{j=1}^n |y_j - \hat{y}_j|$$

Where **n** is the number of observations, and **y_j** and **$\hat{y}_j$** are the actual observation and predicted value, respectively.

Mean Squared Error

34

The **Mean Squared Error (MSE)** is the average of the squared differences between the prediction and the actual observation, regardless of direction.

$$MSE = \frac{1}{n} \sum_{j=1}^n (y_j - \hat{y}_j)^2$$

Where **n** is the number of observations, and **y_j** and **$\hat{y}_j$** are the actual observation and predicted value, respectively.

Root Mean Squared Error

35

The **Root Mean Squared Error (RMSE)** is the average of the squared differences between the prediction and the actual observation, regardless of direction.

$$RMSE = \sqrt{\frac{1}{n} \sum_{j=1}^n (y_j - \hat{y}_j)^2}$$

Where **n** is the number of observations, and **y_j** and **$\hat{y}_j$** are the actual observation and predicted value, respectively.

Regression Quality Metrics

36

$$MAE = \frac{1}{n} \sum_{j=1}^n |y_j - \hat{y}_j| \quad MSE = \frac{1}{n} \sum_{j=1}^n (y_j - \hat{y}_j)^2 \quad RMSE = \sqrt{\frac{1}{n} \sum_{j=1}^n (y_j - \hat{y}_j)^2}$$

Important notes:

- These are the three most common metrics used to **measure accuracy for continuous variables**
- All these metrics express **average model prediction error** (lower values are better)
- They all range from 0 to ∞ and are **indifferent to the direction of errors**
- **MAE** and **RMSE** express the prediction error in the **units of the variable of interest**
- **MSE** and **RMSE** give a relatively **high weight to large errors** by squaring the error
- Therefore, **MSE** and **RMSE** are **more useful when large errors** are particularly **undesirable**

Regression Quality Metrics

37

$$MAE = \frac{1}{n} \sum_{j=1}^n |y_j - \hat{y}_j| \quad MSE = \frac{1}{n} \sum_{j=1}^n (y_j - \hat{y}_j)^2 \quad RMSE = \sqrt{\frac{1}{n} \sum_{j=1}^n (y_j - \hat{y}_j)^2}$$

#	Error	Error	Error ²
1	1	1	1
2	-1	1	1
3	3	3	9
4	3	3	9
		MAE	MSE
		RMSE	
		2	5
		2.24	

#	Error	Error	Error ²
1	0	0	0
2	0	0	0
3	0	0	0
4	10	10	100
		MAE	MSE
		RMSE	
		2.5	25
		5	

Regression Quality Metrics

38

Obtaining the model's **MSE**, **MAE**, and **RMSE** is simple:

```
mean_absolute_error(y_test, predictions)
```

```
14.68592556053812
```

```
mean_squared_error(y_test, predictions)
```

```
1399.7722041242152
```

```
root_mean_squared_error(y_test, predictions)
```

```
37.41352969347072
```

For example, the RMSE tells us that the mean error is 37.41\$



Model Evaluation

Model Validation

40

We must **confirm that the model achieves its intended purpose**. The goal is to assess the model's accuracy and performance based on data the model doesn't know.

Until now, we have been using **hold-out validation**. However, this is a very basic approach. Do you see any problems with this approach?



Model Validation

41

- Hold-out Validation
- Cross Validation
- k-fold Cross Validation
- Leave-one-out Cross Validation
- ...

Cross Validation

42

Cross validation is another **model validation technique**.

The aim is to obtain an accurate measure of how the model will perform in practice.

Essentially, it involves in **dividing the dataset into k folds**. Each time the model runs, **k-1 folds are used for training** and **one fold** (the remaining one) is used as **testing**. This process is repeated until all folds have been used for testing.

The final error metric is based on the mean value of all the error metrics:

$$E = \frac{1}{k} \sum_{i=1}^k E_i$$

Cross Validation

43

How many folds?

Using a **greater number of folds** will lead to a **more accurate model error estimate**. This also results in a **lower bias** and **less overfitting**. However, this comes at a **higher computational cost**.

If we have a **large dataset**, a **smaller k** may be enough since we will have a larger amount of data for training.

If we have a **small dataset**, we may want to **use leave-one-out cross validation** to maximise the amount of data available for training.

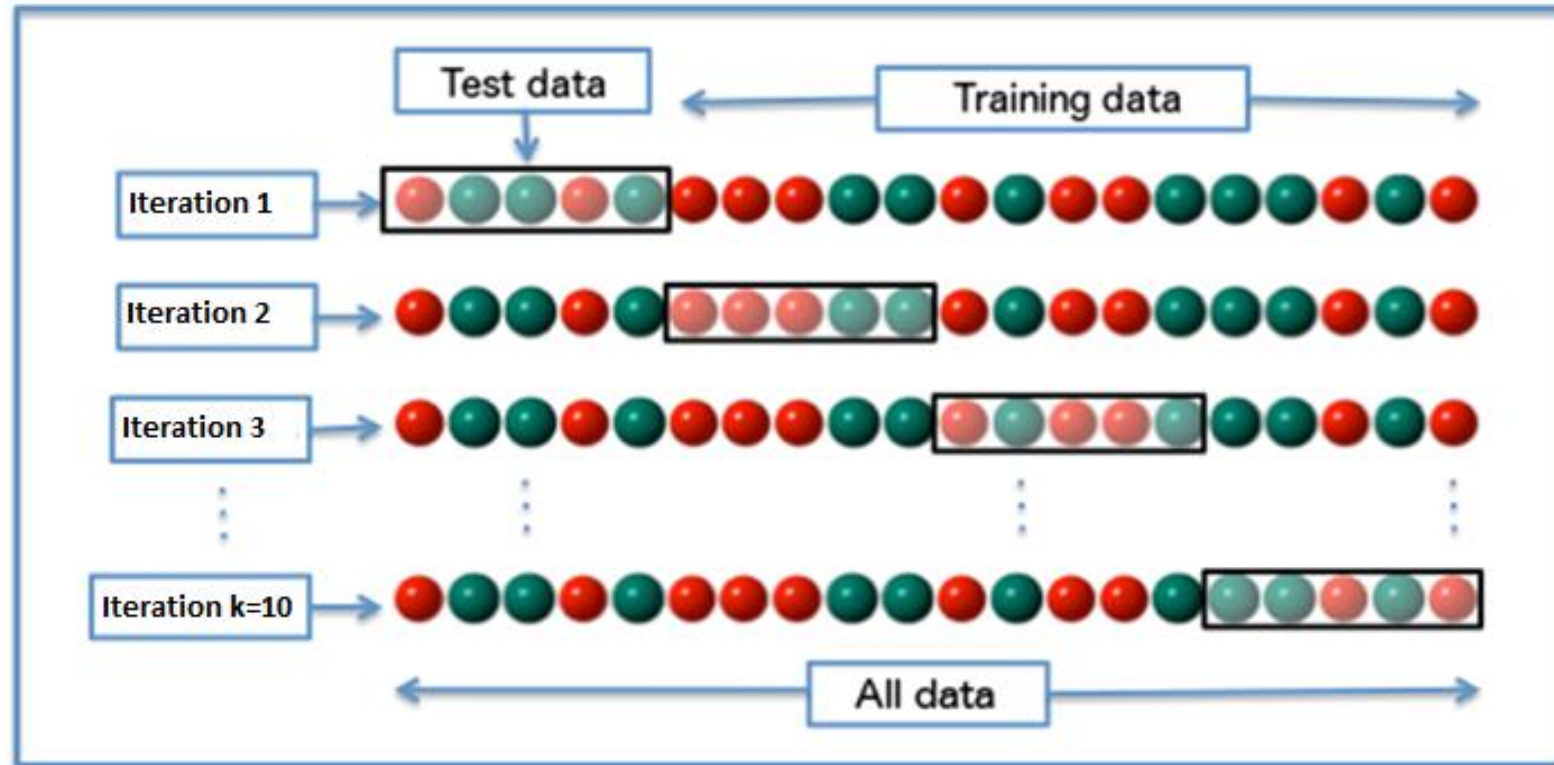
In reality, **k depends on N**.

As a rule of thumb, **k=10**.

Row ID	D Error in %	I Size of Test Set	I Error Count
fold 0	38.281	128	49
fold 1	35.156	128	45
fold 2	44.531	128	57
fold 3	42.969	128	55
fold 4	42.969	128	55
fold 5	41.406	128	53
fold 6	41.406	128	53
fold 7	40.625	128	52
fold 8	41.406	128	53
fold 9	42.52	127	54

k-fold Cross Validation

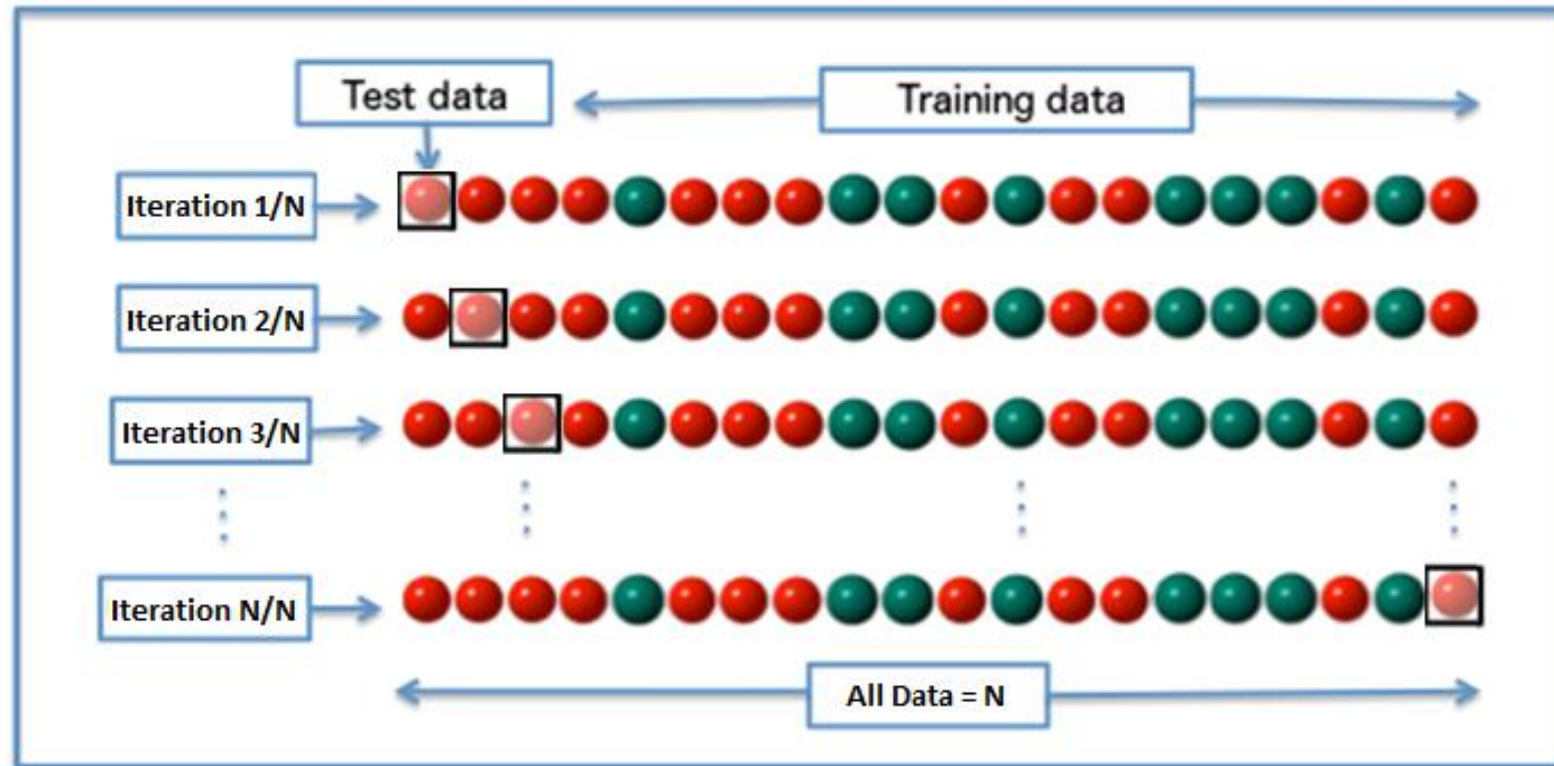
44



Usually, **k=10**

Leave-one-out Cross Validation ($k=N$)

45



The special case of having $k=N$.

Although it is expensive, it is a **good approach** when we have a **small dataset**.

Model Validation

46

Here are some more basic functions and classes that will be need at some point in the future:

```
sklearn.model_selection.train_test_split  
sklearn.model_selection.Kfold  
sklearn.model_selection.LeaveOneOut  
sklearn.model_selection.StratifiedKFold  
sklearn.model_selection.GridSearchCV  
sklearn.model_selection.RandomizedSearchCV  
...
```

Model Validation

47

Using the cross_val_score API:

```
'''
Load CSV
'''
df = pd.read_csv('titanic_dataset.csv')

#Let's start by creating our X (input data) and our y (target feature - the Survived feature)
X = df.drop(['Survived'], axis=1)      #input features - everything except the Survived feature
y = df['Survived'].to_frame()          #target feature

print("USING A DECISION TREE WITH cross_val_score (MEAN ACCURACY)...")
X = X.drop(['Name', 'Sex', 'Age', 'Ticket', 'Cabin', 'Embarked'], axis=1)
clf = DecisionTreeClassifier(criterion='gini', max_depth=10, random_state=2021)
scores = cross_val_score(clf, X, y, cv=10)  ← 10 folds!
print(scores)
print("RESULT: %0.2f accuracy with a standard deviation of %0.2f" % (scores.mean(), scores.std()))

USING A DECISION TREE WITH cross_val_score (MEAN ACCURACY)...
[0.58888889 0.61797753 0.52808989 0.50561798 0.61797753 0.70786517
 0.70786517 0.70786517 0.59550562 0.74157303]
RESULT: 0.63 accuracy with a standard deviation of 0.08
```

Model Validation

48

Or iterating manually with K-fold:

```
print("USING A DECISION TREE WITH MANUAL ITERATION (KFold) and obtaining confusion matrix...")
from sklearn.model_selection import KFold
clf = DecisionTreeClassifier(criterion='gini', max_depth=10, random_state=2021)
scores = []
kf = KFold(n_splits=10)
for train, test in kf.split(X):
    clf.fit(X.loc[train,:], y.loc[train,:])
    score = clf.score(X.loc[test,:], y.loc[test,:])
    scores.append(score)
    y_predicted = clf.predict(X.loc[test,:])
    print("Confusion Matrix:")
    print(confusion_matrix(y.loc[test,:], y_predicted))
    print(score)
print("RESULT: %0.2f accuracy with a standard deviation of %0.2f" % (np.mean(scores), np.std(scores)))
```

USING A DECISION TREE WITH MANUAL ITERATION (KFold) and obtaining confusion matrix...

Confusion Matrix:

```
[[45  6]
 [27 12]]
```

0.6333333333333333

Confusion Matrix:

Confusion Matrix:

```
[[41 15]
 [14 19]]
```

0.6741573033707865

RESULT: 0.65 accuracy with a standard deviation of 0.06



Hands On